Note on the Model Implementation

1. Architecture of the `swaptionPricingG2PP` Function

At the current stage of the implementation, the main difficulty is no longer the derivation of the pricing formula, but rather the correct numerical and structural organization of the code. The objective is therefore to design the `swaptionPricingG2PP` function as a clean orchestration layer built on top of smaller deterministic and analytic helper functions.

The central principle is the following:

> *The main swaption pricing function should not itself be the theorem. It should orchestrate a sequence of precomputed deterministic objects, a Gaussian parameter block, an inner nonlinear solve, and an outer one-dimensional integration.*

This principle is especially important in the present G2++ setting, where the final European swaption formula contains both:

1. an inner scalar nonlinear solve for the critical value $\bar{y}(x)$,
2. an outer one-dimensional Gaussian integral in the variable x.

1.1. Three-layer architecture

The recommended implementation is naturally divided into three layers.

Layer A: low-level scalar/vector kernels

These functions should accept only the precise numerical objects they need. They are direct transcriptions of the theorem and should not recompute model quantities internally.

Typical examples are:

$$h_1(x), \qquad h_{2,i}(x), \qquad \lambda_i(x), \qquad \kappa_i(x),$$

as well as the objective function defining $\bar{y}(x)$ and its derivative.

The reason for this choice is straightforward:

- these objects are called repeatedly inside the quadrature loop,
- they should remain mathematically transparent,
- they should be easy to unit-test independently,
- hidden recomputations must be avoided.

In particular, the function computing $h_1(x)$ should not itself solve for $\bar{y}(x)$, nor should it query the model object to rebuild $\mu_x, \mu_y, \sigma_x, \sigma_y, \rho_{xy}$. Instead, it should take those quantities as explicit inputs.

Layer B: model-dependent swaption builders

These functions may use the full model object `obj`, because they assemble deterministic quantities that depend on the current G2++ specification, the term structure, the volatility buckets, and the instrument conventions.

Typical examples are:

- the instrument block,
- the bond basket block,
- the forward Gaussian parameter block.

These functions should be called once per instrument, not once per quadrature node.

Layer C: orchestration functions

These functions coordinate the pricing logic:

- solving the critical value $\bar{y}(x)$,
- evaluating the integrand at a given node x,
- performing the final numerical quadrature.

At this level, one should avoid embedding algebraic details in the main function body. The role of the orchestration layer is simply to call the correct lower-level components in the correct order.

1.2. Deterministic objects to precompute outside the integrand

For one fixed swaption, all quantities that do not depend on the integration variable x should be precomputed once, before launching the outer quadrature.

These include:

$$P(0,T_0), \qquad \{c_i\}_{i=1}^n, \qquad \{A_i\}_{i=1}^n, \qquad \{B_{x,i}\}_{i=1}^n, \qquad \{B_{y,i}\}_{i=1}^n, \tag{1}$$

together with the Gaussian parameters

$$\mu_x, \qquad \mu_y, \qquad \sigma_x, \qquad \sigma_y, \qquad \rho_{xy}. \tag{2}$$

One should also precompute useful scalar combinations such as

$$\sigma_y\sqrt{1-\rho_{xy}^2}, \qquad \frac{1}{\sigma_x\sqrt{2\pi}}, \qquad \frac{1}{\sigma_x\sqrt{1-\rho_{xy}^2}}, \qquad \frac{1}{\sigma_y\sqrt{1-\rho_{xy}^2}}. \tag{3}$$

The numerical integrand should therefore not perform any model-level work beyond:

1. solving $\bar{y}(x)$,
2. evaluating $h_1(x)$,
3. evaluating $h_{2,i}(x)$,
4. evaluating $\lambda_i(x)$,
5. evaluating $\kappa_i(x)$,
6. assembling the final scalar integrand.

1.3. Separation of the critical-value solver

A key architectural point is that the nonlinear solve for $\bar{y}(x)$ must remain separate from the theorem objects that depend on it.

The natural sequence is

$$x \longrightarrow \bar{y}(x) \longrightarrow h_1(x) \longrightarrow h_{2,i}(x).$$

This implies in particular:

- the solver for $\bar{y}(x)$ is a dedicated numerical routine;
- $h_1(x)$ should use the already-computed value $\bar{y}(x)$;
- $h_2(x)$ should ideally use $h_1(x)$ plus the additional deterministic shift.

Thus, h_2 should not be treated as a second full theorem object. It is merely

$$h_{2,i}(x) = h_1(x) + B_y(T_0,T_i)\,\sigma_y\sqrt{1-\rho_{xy}^2}. \tag{4}$$

1.4. Numerical integration strategy in MATLAB

The outer integral is formally defined on the whole real line. In practice, the recommended strategy is to standardize the Gaussian variable:

$$x = \mu_x + \sigma_x z. \tag{5}$$

Under this transformation, the integral becomes one with respect to the standard normal density in z. The numerical integration may then be performed on a bounded interval

$$z \in [-L, L], \tag{6}$$

with a first practical choice such as

$$L = 6 \quad \text{or} \quad L = 7.$$

This approach is particularly attractive here because:

- the library already provides a bounded-interval Gaussian quadrature;
- the integrand is not merely a polynomial correction of a Gaussian kernel;
- each node evaluation requires the inner nonlinear solve for $\bar{y}(x)$, so implementation robustness is more important than formal elegance.

Consequently, a bounded quadrature after Gaussian standardization is an entirely reasonable first implementation, and often preferable to jumping immediately to Gauss–Hermite quadrature.

1.5. Warm-start strategy for the inner nonlinear solver

The most important practical numerical advice is to exploit the smooth dependence of $\bar{y}(x)$ on x.

Indeed, the quadrature nodes $x_1, x_2, \ldots$ are ordered and typically close to one another. Hence the critical value $\bar{y}(x)$ varies smoothly as a function of x. This suggests the following strategy:

1. solve $\bar{y}(x_1)$ using a generic initial guess;
2. use $\bar{y}(x_1)$ as the initial guess for $\bar{y}(x_2)$;
3. continue recursively along the ordered quadrature nodes.

This warm-start policy can significantly reduce the computational burden of the swaption pricing routine.

1.6. Recommended package structure

It is advantageous to conceptually group the deterministic quantities into a single package:

$$\mathcal{P} = \{P(0,T_0),\ \{c_i\}_{i=1}^n,\ \{A_i\}_{i=1}^n,\ \{B_{x,i}\}_{i=1}^n,\ \{B_{y,i}\}_{i=1}^n,\ \mu_x, \mu_y, \sigma_x, \sigma_y, \rho_{xy}\}\,. \tag{7}$$

Then one may think of the numerical architecture as:

$$\texttt{solveYBar}(x, \mathcal{P}), \qquad \texttt{swaptionIntegrandX}(x, \mathcal{P}), \qquad \texttt{swaptionPricingG2PP}(\mathcal{P}).$$

Such a structure avoids the propagation of too many loose scalar arguments and keeps the pricing logic readable.

1.7. Immediate implementation roadmap

At the current stage, the next coding steps should follow the sequence:

1. finalize the Gaussian parameter block;
2. finalize the swaption package by combining:
   - the instrument block,
   - the bond basket block,
   - the Gaussian block;
3. define the low-level theorem kernels:
$$h_1, \quad h_2, \quad \lambda_i, \quad \kappa_i;$$
4. finalize the scalar objective function defining $\bar{y}(x)$ and its derivative;
5. finalize the nonlinear solver for $\bar{y}(x)$;
6. define the scalar integrand in the variable x;
7. finally, define the public pricing function `swaptionPricingG2PP` as an orchestration routine around bounded Gaussian quadrature.

1.8. Guiding MATLAB snippets

The following snippets are not meant to be complete implementations. Their role is only to clarify the intended architecture and the separation of responsibilities between low-level kernels, model-dependent builders, and orchestration functions.

Low-level scalar/vector kernel style

These functions should receive only the precise numerical objects they need. They should not trigger hidden recomputations of model quantities.

```
function output = h1Function(obj, xVal, yBarVal, mu_x, mu_y, sigma_x, sigma_y, rho_xy)
    arguments
        obj (1,1) GaussianTwoFactors
        xVal (1,1) double
        yBarVal (1,1) double
        mu_x (1,1) double
        mu_y (1,1) double
        sigma_x (1,1) double
        sigma_y (1,1) double
        rho_xy (1,1) double
    end

    % purely local computation, no hidden recomputation
end
```

Using $\bar{y}(x)$ outside of h_1

The critical value $\bar{y}(x)$ should be solved first. The result should then be passed explicitly to the theorem-level helper functions.

```
function output = swaptionIntegrandX(obj, xVal, swaptionPackage, yInitialGuess)
    arguments
        obj (1,1) GaussianTwoFactors
        xVal (1,1) double
        swaptionPackage struct
        yInitialGuess (1,1) double
    end

    % 1) solve yBar(x)
    % 2) compute h1
    % 3) compute h2_i vector
    % 4) compute lambda_i vector
    % 5) compute kappa_i vector
    % 6) assemble scalar integrand
end
```

Model-level assembly block style

The Gaussian block is model-dependent. It is therefore natural for this type of helper to use the full model object.

```
function output = getSwaptionGaussianBlock(obj, currMaturity)
    arguments
        obj (1,1) GaussianTwoFactors
        currMaturity (1,1) double
    end

    % build:
    %   mu_x, mu_y
    %   sigma_x, sigma_y
    %   rho_xy
    %
    % swaption specialization:
    %   startDate = 0
    %   endDate   = currMaturity
    %   tMaturity = currMaturity
end
```

Outer quadrature wrapper style

The public pricing function should remain short. Its role is to coordinate the blocks rather than to contain all the formula details inline.

```
function output = swaptionPricingG2PP(obj, strike, currMaturity, ulSwapSteps, args)
    arguments
        obj (1,1) GaussianTwoFactors
        strike (1,1) double
        currMaturity (1,1) double
        ulSwapSteps (1,:) double
        args.direction (1,:) char = 'PAY'
        args.notional (1,1) double = 1e6
    end

    % 1) build instrument
    % 2) build bond basket
    % 3) build gaussian block
    % 4) create swaption package
    % 5) standardize x = mu_x + sigma_x z
    % 6) integrate on bounded interval [-L, L]
end
```

Warm-start logic for the inner solver

Because $\bar{y}(x)$ varies smoothly with x, the previous root should be recycled as the initial guess for the next quadrature node.

```
yGuess = 0.0;
for k = 1:nNodes
    xVal = xNodes(k);

    % solve yBar(xVal) with previous root as new initial guess
    yBarVal = ...;
    yGuess = yBarVal;

    % evaluate integrand
    integrandVals(k) = ...;
end
```

1.9. Conclusion

The implementation principle may be summarized as follows:

> *All model-dependent deterministic objects should be assembled once, outside the quadrature loop; all theorem-level kernels should operate only on explicit scalar or vector inputs; the main pricing function should remain a short orchestration layer around the instrument block, the bond basket block, the Gaussian block, the inner nonlinear solver, and the bounded Gaussian quadrature.*

This architecture is both numerically robust and consistent with the implementation philosophy already present in the Hull–White code base.